ICS-OS Lab 04: Lottery Scheduling

Objectives

At the end of this activity, you should be able to:

1. trace how ICS-OS selects the next process to run, from the timer interrupt down to the scheduler;
2. explain the lottery scheduling algorithm (Waldspurger and Weihl, 1994) and how tickets achieve *proportional-share* CPU allocation;
3. implement lottery scheduling by modifying the ICS-OS default scheduler; and
4. verify experimentally that the CPU time received by threads is proportional to their ticket holdings.

1 Introduction

The default ICS-OS scheduler is *round-robin*: every runnable process gets an equal share of the CPU. In this lab you will replace it with a *lottery scheduler*, which gives each process a number of **tickets** and holds a raffle every time the CPU must be handed to someone. A process holding three times the tickets of another wins about three times as often, and so receives about three times the CPU time.

2 Prerequisites

To proceed with this lab, you should have completed `Lab 01` and `Lab 02`. Build the kernel in the Docker build environment described in Lab 01 and the README. Most of the commands below are run relative to the `$ICSOS_HOME` directory. Update your local copy of the source code and create a new branch for this lab with the commands below.

```
$cd $ICSOS_HOME
$git checkout master
$git pull
$git checkout -b starter/lottery-scheduling-lab
$git branch    #to check the current branch
```

Unless stated otherwise, all source paths in this document are under `ics-os/kernel/`.

3 Deliverables and Credit

Perform the tasks below and capture screen shots while you do them. Answer all questions. Submit a PDF file containing the screen shots with captions, your answers to the questions, and your modified `scheduler.c` (or the output of `git diff`). Do not forget to put your name and laboratory section. Total is **85 points**.

4 Background

How scheduling works in ICS-OS

Study the following parts of the kernel (These have been discussed in the handout).

 JACHermocilla | CC BY-NC-SA 4.0

- `process/process.h` defines `PCB386`, the process control block. Each PCB sits in a **circular doubly-linked list** (the `next` and `before` pointers). Note these fields: `status` (the `PS_ATTB_BLOCKED` bit marks a blocked process), `waiting` (a countdown for sleeping processes), `totalcputime` (incremented on every tick the process runs), and `priority`, which the default scheduler **never uses**. You will reuse `priority` as the ticket count.
- `process/process.c`, function `taskswitcher()` (around line 1315) runs on every timer interrupt, 100 times per second. It asks the current scheduler for the next process and performs the context switch with `ps_switchto()`.
- `process/scheduler.c` is the default scheduler. The function `scheduler(PCB386 *lastprocess)` implements round-robin: starting from `lastprocess->next`, it returns the first process that is neither blocked nor waiting, decrementing `waiting` as it passes sleepers.
- `console/console.c` is the kernel shell. It already contains the `tickets` command (see below). You do **not** modify this file.

Study `process/scheduler.c` carefully before writing any code. This is where you will place all your code. You will not change other files.

Lottery scheduling

Lottery scheduling gives each process a number of *tickets*. On every scheduling decision the kernel holds a lottery:

1. Sum the tickets of all *runnable* processes. Call the sum `totaltickets`.
2. Draw a uniform random number `winner` in [0, `totaltickets`).
3. Walk the ready queue, accumulating the tickets of each runnable process into a counter. The first process for which `counter > winner` wins the CPU for this quantum.

Over many quanta, a process holding three times the tickets of another receives about three times the CPU time.

Random number generation (provided)

the ICS-OS kernel has no `rand()`. You must implement your own pseudo-random number generator. A *xorshift32* generator (Marsaglia, 2003) is already provided for you:

```
//seed/state for the pseudo-random number generator
static DWORD lottery_state = 2463534242UL;

//xorshift32 pseudo-random number generator (Marsaglia, 2003).
//The kernel has no rand(), so the scheduler brings its own.
static DWORD lottery_rand(){
        lottery_state ^= lottery_state << 13;
        lottery_state ^= lottery_state >> 17;
        lottery_state ^= lottery_state << 5;
        return lottery_state;
};
```

 JACHermocilla | CC BY-NC-SA 4.0

The `tickets` command (provided)

The kernel shell in your starter code already has a command that sets the tickets of a process:

```
tickets <pid> <ntickets>
```

Here's the implementation in `kernel/console/console.c`:

```
if (strcmp(u,"tickets") == 0){      //-- Sets the lottery tickets of a process. Args: <pid> <t
        char *p1 = strtok(0," ");
        char *p2 = strtok(0," ");
        if (p1!=0 && p2!=0){
                PCB386 *proc = ps_findprocess(atoi(p1));
                if (proc != (PCB386*)-1){
                        proc->priority = atoi(p2);
                        printf("tickets: pid %d now holds %d ticket(s).\n", atoi(p1), atoi(p2)
                }else
                        printf("tickets: process %d not found.\n", atoi(p1));
        }else
                printf("usage: tickets <pid> <ntickets>\n");
}
```

It finds the process with `ps_findprocess(pid)` and stores `ntickets` in its `priority` field, the same field your scheduler will read. It prints a usage message when an argument is missing and an error when the pid does not exist. **Note that we used the priority field of the PCB as the placeholder for the tickets assigned to a process**.

Returning the number of tickets held by a process (provided)

This is a helper function that returns the number of tickets held by a process. For processes that were not allocated tickets (such as the startup processes and kernel threads), a default number of tickets is returned. Call this when implementing the lottery scheduler.

```
//Default number of tickets given to a process that has not been
//assigned any (i.e. priority == 0)
#define DEFAULT_TICKETS 10

//returns the number of lottery tickets held by a process.
//The unused PCB386.priority field is reused as the ticket count.
static DWORD lottery_tickets(PCB386 *ptr){
        if (ptr->priority == 0)
                return DEFAULT_TICKETS;
        return ptr->priority;
};
```

 JACHermocilla | CC BY-NC-SA 4.0

The measuring tool: `lotto`

The user application `contrib/lotto` is provided for experimental evaluation. Do **not** modify it. Since the code is quite long, it is not included in this text. It is available in the given starter code.

```
lotto.exe <ticketsA> <ticketsB> [seconds]
```

`lotto.exe` starts two CPU-bound threads, A and B. It gives them the tickets you ask for by running the `tickets` command on them, lets them compete for `[seconds]` (default 10), and then prints how much work each one completed and what share of the total that is. If your scheduler is correct, the share of A is close to `ticketsA`/(`ticketsA` + `ticketsB`).

While `lotto.exe` runs, the shell is busy and does not accept typing. Wait for the prompt to come back.

The "seconds" are kernel seconds. Under an emulator such as QEMU, the kernel clock can run slower than your wall clock, so a 10-second run may take noticeably longer to finish.

5 Tasks

Task 1: Code Tracing (15 points)

No coding is needed here. Read the source files described in the Background and answer the questions.

QUESTIONS(Provide screen shots to support your answer):

1. Who calls `scheduler()`, and how often? (Hint: follow `taskswitcher()` and the timer interrupt.)
2. Why does the round-robin loop decrement `ptr->waiting` while scanning? What would happen to sleeping processes if it did not?
3. The comment says the scheduler "assumes that at least one process is not blocked or waiting." Why is this assumption safe in ICS-OS? (Hint: what do the kernel process and its threads do?)

Task 2: Baseline, Measure Round-Robin (5 points)

Before you change anything, build the stock kernel (one with default scheduler) and measure it. Run the commands below (the first group inside the Docker build environment).

```
$docker compose --remove-orphans run ics-os-build
(in container)# cd /home/ics-os && make clean && make
(in container)# cd contrib/lotto && make && make install && cd ../..
(in container)# exit
$sudo make floppy
$qemu-system-i386 -fda ics-os-floppy.img
```

At the ICS-OS prompt run the two commands below.

```
[1]% lotto.exe 10 10 20
[2]% lotto.exe 30 10 20
```

 JACHermocilla | CC BY-NC-SA 4.0

`lotto.exe` will report that thread A and thread B hold the tickets you asked for, but the stock scheduler ignores them. Capture a screen shot and record the shares printed. Round-robin ignores tickets, so **both** runs should split the work close to 50% / 50%, even though the second run gives A three times the tickets of B.

Task 3: Implement Lottery Scheduling (35 points)

Edit `process/scheduler.c`. This is the only file you modify. Replace the round-robin logic in `scheduler()` with lottery scheduling:

1. Add a `lottery_rand()` xorshift32 PRNG (a `static DWORD` state variable with a non-zero seed).
2. Add a helper `lottery_tickets(PCB386 *ptr)` that returns the process's ticket count. Use the unused `priority` field as the ticket count. If it is `0` (the default for every process), return `DEFAULT_TICKETS` (use 10).
3. **Pass 1:** traverse the circular list exactly once, starting at `lastprocess->next`. Decrement `waiting` on sleepers, and sum the tickets of runnable processes (not `PS_ATTB_BLOCKED`, not `waiting`).
4. If `totaltickets == 0`, fall back to the original round-robin loop, so that the kernel still behaves if nothing is runnable on this tick.
5. **Pass 2:** draw `winner = lottery_rand() % totaltickets`, walk the list again accumulating the tickets of runnable processes, and return the winner.
6. Update the strings in `ps_scheduler_install()` (`hdr.name`, `hdr.description`, and the startup `printf`) so that they say lottery instead of round-robin.

Constraints. Do **not** modify `PCB386`, `taskswitcher()` or `console.c`. Your whole change lives in `scheduler.c`. Interrupts are already disabled when `scheduler()` runs, so no extra locking is needed. Your function must terminate, and it must never return a blocked or waiting process.

Task 4: Build and Experiment (20 points)

Rebuild the kernel and the floppy image as in Task 2 and boot it. Only `scheduler.c` has changed, so you do not need to rebuild `lotto`.

1. The system must boot to the shell and `ps` must work. If your scheduler is wrong, the system usually hangs during boot or at the first command.
2. Run the three commands below. Run each at least **three times**.

```
[1]% lotto.exe 10 10 20
[2]% lotto.exe 30 10 20
[3]% lotto.exe 10 40 20
```

3. Record the share that each run reports and compare it with the share the program lists as expected.
4. Put your results in a table with these columns: tickets, expected share, measured share for each trial, and the average. Add the Task 2 baseline results for comparison.

Here is a sample of what a short run (`lotto.exe 30 10 8`) prints. Your numbers will differ.

```
lotto: thread A = pid 21, thread B = pid 22
tickets: pid 21 now holds 30 ticket(s).
tickets: pid 22 now holds 10 ticket(s).
lotto: running for 8 seconds...
tickets   A = 30   B = 10
```

 JACHermocilla | CC BY-NC-SA 4.0

```
work      A = 460331   B = 172394   (units of 1000 loops)
share     A = 72%   B = 27%
expected  A = 75%   B = 25%
```

The two shares can add up to 99% because each is rounded down.

6 Report Questions (10 points)

QUESTIONS:

1. Your measured shares will not be exactly the expected ones. Give two reasons. (Think: how many lotteries are held in a 20-second run at 100 ticks per second, and which other processes hold tickets?)
2. The baseline in Task 2 ignored tickets, and the lottery kernel did not. Which lines of your `scheduler()` are responsible for the difference?
3. Lottery scheduling is probabilistic. Describe *stride scheduling*, its deterministic counterpart, and one advantage it has.
4. What happens if a malicious user program could set its own tickets to a huge value? How do real systems guard proportional-share mechanisms?
5. Why is it acceptable for the scheduler to use a weak, non-cryptographic PRNG here?

7 Reflection

Write some realizations and questions that crossed your mind while doing this lab.

Reference

C. A. Waldspurger and W. E. Weihl. *Lottery Scheduling: Flexible Proportional-Share Resource Management.* OSDI 1994.

 JACHermocilla | CC BY-NC-SA 4.0